

Module 2

Isaac Sim, USD/URDF, Walker S2 và baseline architecture

Tài liệu đào tạo nội bộ của BA x AILAB@UEL

Mục tiêu của module: Sau module này, sinh viên có thể mở và đọc một scene Isaac Sim, hiểu các khái niệm USD/URDF/STL ở mức đủ dùng, xác định link/joint/end-effector/camera trong model Walker S2, đọc cấu trúc baseline HRC2026 và lập checklist config cho từng task.

Thông tin	Nội dung
Thời lượng khuyến nghị	1.5 ngày học tập trung, tương đương 8-12 giờ tự học có hướng dẫn.
Học chung cho	Tất cả sinh viên trong training camp; Platform guild và Manipulation guild cần đi sâu hơn.
Đầu ra bắt buộc	Scene map, robot structure note, code reading note, config checklist.
Không yêu cầu ở Module 2	Chưa yêu cầu sinh viên giải IK, viết motion primitive hoặc train policy. Các phần này thuộc Module 3, 6, 7, 8.

Mục lục định hướng

1. Vị trí của Module 2 trong pipeline HRC2026
2. Chuẩn đầu ra và cách tự học
3. Isaac Sim ở mức sinh viên cần nắm
4. OpenUSD: stage, prim, path, layer, reference, Xform
5. USD, URDF và STL: khác nhau ở đâu và dùng khi nào
6. Đọc model Walker S2: link, joint, camera, end-effector
7. Baseline HRC2026: kiến trúc thư mục và luồng chạy
8. Config theo task: đọc đúng trước khi sửa
9. Quy trình code reading và reverse engineering baseline
10. Các lỗi thường gặp khi đọc scene/model/baseline
11. Lab thực hành từng bước
12. Mini-project cuối module và rubric chấm điểm
13. Checklist hoàn thành Module 2
14. Tài liệu tham khảo mở/chính thức

1. Vị trí của Module 2 trong pipeline HRC2026

Module 1 giúp sinh viên có môi trường chạy được: Linux, Git, Docker, GPU/CUDA, Isaac Sim và smoke test. Module 2 trả lời câu hỏi tiếp theo: khi môi trường đã chạy, chúng ta đang chạy cái gì, scene nằm ở đâu, robot được mô tả như thế nào, camera và vật thể nằm ở đường dẫn nào, baseline nối các thành phần đó lại ra sao.

Nếu không học Module 2 cẩn thận, nhóm có thể gặp các lỗi rất tốn thời gian: gọi sai prim path, nhầm USD với URDF, không biết joint nào thuộc cánh tay, không biết camera nào đang được dùng, hoặc sửa nhầm config. Vì vậy, Module 2 là cầu nối giữa môi trường phát triển và các module sau về geometry, perception, manipulation, data và evaluation.

Từ Module 2 sang module sau	Vì sao cần Module 2
Module 3 - frames, transforms, kinematics	Cần biết các frame/prim/link/camera nào tồn tại trong scene để mới tính được transform.
Module 4-5 - camera, RGB-D, perception	Cần biết camera prim, output sensor, object prim path, vật nào là target.
Module 6 - motion primitives	Cần biết end-effector, gripper, joint names, articulation root và vùng thao tác.
Module 7-8 - data/policy	Cần biết observation/action đến từ đâu, data được ghi theo entypoint nào.
Module 10-11 - evaluation/submission	Cần biết config task, script chạy và asset dependency để package tái lập được.

Nguyên tắc: Module 2 không nhằm biến sinh viên thành chuyên gia Isaac Sim. Mục tiêu là đủ năng lực đọc scene, đọc robot model, đọc baseline và viết tài liệu kỹ thuật có thể dùng lại cho cả nhóm.

2. Chuẩn đầu ra và cách tự học

Mã	Chuẩn đầu ra sau Module 2
M2-CLO1	Giải thích được vai trò của Isaac Sim trong vòng mô phỏng HRC2026 và phân biệt scene, physics, robot, camera, asset, task config.
M2-CLO2	Giải thích được các khái niệm OpenUSD cơ bản: stage, prim, prim path, Xform, Mesh, Camera, Light, layer, reference.
M2-CLO3	Phân biệt được USD, URDF và STL: file nào mô tả scene, file nào mô tả robot, file nào chỉ là mesh hình học.
M2-CLO4	Tạo được robot structure note cho Walker S2: root/base, waist, arms, grippers, end-effector candidates, camera/sensor, joint groups.
M2-CLO5	Đọc được baseline architecture ở mức entryptoint, folder, config, data flow và pipeline chạy.
M2-CLO6	Lập được checklist config cho 4 task và xác định những thông tin cần thiết trước khi sửa code.
Khung tự học 8-12 giờ	Hoạt động
Giờ 1	Đọc nhanh toàn bộ tài liệu, đánh dấu thuật ngữ chưa hiểu.
Giờ 2-3	Mở Isaac Sim/baseline, quan sát Stage, Viewport, Properties và object paths.
Giờ 4	Đọc USD/URDF/STL; chạy script liệt kê prim hoặc đọc URDF bằng Python.
Giờ 5-6	Đọc Walker S2 model và điền robot structure note.
Giờ 7-8	Đọc baseline: entryptoints, configs, folders, luồng observe-action-log.
Giờ 9-12	Hoàn thiện mini-project: scene map, code reading note, config checklist, README.

Cách học hiệu quả: Không chỉ đọc tài liệu. Mỗi khái niệm phải được kiểm chứng bằng một object path, một file config, một dòng code, một log hoặc một ảnh chụp màn hình. Khi chưa chắc, ghi lại câu hỏi vào issue thay vì sửa code ngẫu nhiên.

3. Isaac Sim ở mức sinh viên cần nắm

Isaac Sim là môi trường mô phỏng robot dựa trên NVIDIA Omniverse và OpenUSD. Trong cuộc thi, Isaac Sim đóng vai trò là “sân thi ảo”: tạo scene, nạp robot Walker S2, mô phỏng vật lý, camera/sensor, hành động robot, đồng thời là nơi hệ thống đánh giá tự động chạy model của đội.

Thành phần trong Isaac Sim	Sinh viên cần hiểu gì	Sản phẩm cần ghi lại
Stage	Cây scene chứa toàn bộ prim: world, robot, table, box, conveyor, foam, camera, light.	Danh sách prim path quan trọng.
Viewport	Cửa sổ quan sát 3D; dùng để kiểm tra object có đúng vị trí, không bị rơi xuyên bàn, camera nhìn đúng hướng.	Ảnh chụp scene hoặc mô tả layout.
Properties	Bảng xem thuộc tính prim: transform, material, physics, collision, camera parameters.	Ghi lại transform/camera/physics quan trọng.
Timeline/Play	Điều khiển mô phỏng vật lý. Khi bấm Play, gravity, collision, joints và controllers mới hoạt động.	Ghi lại hiện tượng khi Play/Pause.
Extensions	Các module mở rộng: URDF importer, ROS bridge, camera/sensor, examples.	Extension nào cần bật cho task.
Console/Log	Nơi đọc lỗi asset path, missing extension, physics warning, Python exception.	File log hoặc lỗi chính đã gặp.

3.1. Scene, physics, camera và articulation

Trong HRC2026, sinh viên cần đọc scene theo bốn lớp: hình học hiển thị, mô phỏng vật lý, sensor/camera và robot articulation. Không nên chỉ nhìn vật thể bằng mắt trong viewport rồi kết luận scene đúng; cần kiểm tra cả thuộc tính physics và path trong Stage.

Lớp	Câu hỏi cần tự hỏi
Hình học/asset	Vật thể nằm ở path nào? Là Mesh hay Xform? Có transform đúng không? Có đang bị scale sai không?
Physics/collision	Vật thể có collider không? Có rigid body không? Bàn/foam/box có static collider không? Có vật nào rơi xuyên qua bàn không?
Camera/sensor	Camera nào được policy/perception dùng? RGB/depth lấy từ đâu? Camera có nhìn thấy vật

	target không?
Robot articulation	Robot root ở đâu? Joint names là gì? Cánh tay trái/phải gồm joint nào? Gripper có joint hoặc control interface nào?

3.2. Những thao tác GUI cần biết

- ☐ Mở scene từ baseline hoặc từ asset folder.
- ☐ Tìm một prim trong Stage bằng tên hoặc bằng cách chọn trực tiếp trong Viewport.
- ☐ Copy prim path của robot, object, camera, bin, conveyor, foam/carton.
- ☐ Xem transform translation/rotation/scale của một prim.
- ☐ Bật/tắt Play để kiểm tra physics.
- ☐ Mở log/console để tìm lỗi asset path, missing material, warning physics.
- ☐ Chụp màn hình hoặc ghi lại layout scene làm scene map.

Mẫu ghi scene map

```
# Gợi ý ghi scene map dạng text
Scene: Task 1 - Desktop Sorting
Robot root: /World/WalkerS2 hoặc path thực tế trong scene
Table: /World/Table hoặc path thực tế
Camera(s): /World/.../camera...
Objects: /World/.../workpiece_*
Bins: /World/.../bin_A, /World/.../bin_B
Notes: camera nhìn từ ..., vật random pose, bins ở phía ...
```

Lưu ý quan trọng: Không được giả định tên path. Mỗi phiên bản asset/baseline có thể thay đổi tên prim, folder hoặc config. Sinh viên phải kiểm tra trực tiếp trong scene hoặc code hiện tại.

3.3. Isaac Sim là một stack, không chỉ là phần mềm 3D

Để dùng Isaac Sim hiệu quả trong HRC2026, sinh viên nên nhìn Isaac Sim như một stack nhiều lớp. Khi có lỗi, phải xác định lỗi thuộc lớp nào thay vì chỉ nói chung chung “Isaac Sim bị lỗi”. Bảng dưới đây giúp định vị lỗi nhanh hơn.

Lớp trong Isaac Sim stack	Vai trò	Ví dụ lỗi thường gặp trong HRC2026
Omniverse Kit / Extensions	Nền ứng dụng, hệ thống extension, UI, Script Editor, Examples Browser.	Extension URDF importer/camera/ROS chưa bật; menu hoặc ví dụ không xuất hiện.
OpenUSD scene graph	Mô tả scene bằng Stage, Prim, Layer, Reference, Asset Path.	Sai prim path, thiếu reference, scene load nhưng object không xuất hiện.
Asset layer	Robot model, mesh, material, texture, task assets.	Git LFS chưa tải mesh; vật bị mất texture; file STL/USD không đúng path.

PhysX physics	Gravity, rigid body, collider, contact, joints, articulation.	Object rơi xuyên bàn; robot rung; joint không chuyển động; collision quá nặng.
RTX rendering	Render ảnh RGB, ánh sáng, vật liệu, camera view.	Camera đen, ảnh thiếu object, FPS thấp, GPU memory cao.
Sensors & Replicator	Camera RGB/depth, segmentation, bounding boxes, synthetic data.	Depth/segmentation không có vì chưa bật annotator hoặc object thiếu semantic label.
Robot control APIs	Articulation, joint commands, controllers, OmniGraph/Python.	Lệnh joint sai thứ tự, robot chưa initialize, command gửi trước khi simulation play.
Application layer / baseline	Task config, teleop, data logging, policy inference, evaluation.	Chạy nhầm config, log sai folder, policy đọc observation khác format.

3.4. Ba workflow chính: GUI, Extension, Standalone Python

Isaac Sim có ba workflow chính. Trong cuộc thi, cả ba đều hữu ích nhưng dùng ở các thời điểm khác nhau. GUI giúp học scene và debug trực quan; Extension/Script Editor giúp thử nhanh API khi Isaac Sim đang mở; Standalone Python/headless phù hợp để chạy evaluation hoặc training pipeline có thể tái lập.

Workflow	Khi nào dùng	Ưu điểm	Rủi ro / lưu ý
GUI	Học giao diện, mở scene, kiểm tra Stage/Viewport/Properties, attach sensor, inspect object.	Trực quan, phù hợp sinh viên mới, dễ chụp ảnh và lập scene map.	Không tái lập hoàn toàn nếu chỉ thao tác bằng tay mà không ghi lại config/script.
Extension / Script Editor	Test nhanh Python snippets, kiểm tra prim path, tạo tool nhỏ, xem hot reload.	Không cần restart app, phù hợp debug nhanh.	Python chạy bất đồng bộ với rendering/physics; cần hiểu callback/async khi can thiệp simulation.
Standalone Python / Headless	Batch evaluation, data generation, inference, chạy nhiều seed, chạy qua Docker/CI.	Có thể kiểm soát physics/render step, tái lập và tự động hóa tốt.	Khó debug bằng mắt; cần log tốt và cần hiểu lifecycle của SimulationApp.

Khuyến nghị cho HRC2026: Ngày đầu học Isaac Sim nên dùng GUI để hiểu Stage và scene. Khi đã có scene map, mọi thứ cần chuyển dần sang script/headless để phục vụ evaluation và submission.

3.5. Vòng đời một simulation episode trong Isaac Sim

Một episode không bắt đầu từ lúc policy dự đoán action. Nó bắt đầu từ việc app/scene/physics/robot/sensor được khởi tạo đúng. Sinh viên cần hiểu lifecycle này để tránh các lỗi như

đọc camera quá sớm, gửi joint command khi robot chưa initialize hoặc tính metric khi scene chưa reset xong.

Bước	Việc xảy ra	Điểm cần kiểm tra
1. Launch app	Khởi động Isaac Sim GUI hoặc headless.	Đúng version, đúng GPU, đúng cache/log folder.
2. Load config	Đọc YAML/JSON/args của task.	Task name, scene path, robot path, camera path, policy path.
3. Load USD stage/assets	Nạp scene, robot, table, objects, references.	Không missing asset; prim path tồn tại.
4. Initialize physics/world	Tạo physics scene, gravity, timestep, colliders.	Vật không rơi xuyên bàn; collider hợp lý.
5. Initialize robot/articulation	Robot wrapper, joint view, controller, gripper.	Joint names/order đúng; robot đã initialize.
6. Initialize sensors	Camera/render products, RGB/depth/segmentation.	Ảnh có object target; depth/metadata đúng.
7. Reset task	Spawn workpieces, đặt bin/foam/carton, reset robot pose.	Random seed/log, object count, target positions.
8. Control loop	observe -> plan/policy -> action -> physics step -> log.	Không block quá lâu; action hợp lệ; log đủ.
9. Success/evaluation	Tính thành công, thời gian, sai số, lỗi.	Metric khớp mục tiêu; failure reason rõ.
10. Cleanup	Lưu log, đóng app nếu batch run.	Không leak GPU memory; checkpoint/log có version.

3.6. UI cần nắm: nhìn Isaac Sim bằng mắt nhưng ghi bằng evidence

UI element	Sinh viên cần làm được	Evidence cần lưu vào báo cáo/lab
Menu Bar	Mở scene, import asset/URDF, bật extension, truy cập examples.	Tên menu hoặc ảnh chụp thao tác nếu cần.
Viewport	Quan sát scene 3D, đổi camera view, kiểm tra object và robot.	Ảnh viewport có annotation hoặc mô tả layout.
Main Toolbar / Play-Stop	Bắt đầu/dừng simulation, move/rotate/scale selected prim.	Ghi rõ đã kiểm tra khi Play và Pause.
Stage panel	Tìm prim, copy prim path, phân	Danh sách prim path chính xác.

	biệt root/group/mesh/camera.	
Property panel	Xem transform, visibility, material, physics/collider, camera parameters.	Bảng thông số quan trọng.
Content/Asset Browser	Tìm sample assets, task assets, robot model, texture/mesh.	Asset path và nguồn asset.
Layer panel	Nhận biết scene chính và referenced layers.	Layer nào là scene chính, layer nào là asset.
Extensions	Bật URDF importer, Replicator, ROS bridge, examples nếu cần.	Tên extension và trạng thái enabled.
Console/Output Log	Đọc warning/error, copy stack trace, tìm missing file/path.	Trích log lỗi và cách tái hiện.

3.7. Physics trong Isaac Sim: các khái niệm tối thiểu

Trong HRC2026, robot thao tác với vật thể công nghiệp, bàn, băng chuyền, foam và carton. Vì vậy physics không phải phần phụ. Ngay cả một pipeline AI tốt cũng thất bại nếu collider sai, vật rơi xuyên bàn hoặc robot gửi action trước khi articulation sẵn sàng.

Khái niệm physics	Ý nghĩa	Kiểm tra nhanh
Rigid body	Vật động chịu lực, va chạm, trọng lực.	Workpiece có rigid body không?
Static collider	Vật đứng yên nhưng có bề mặt va chạm.	Table/bin/foam/carton có collider không?
Visual mesh vs collision mesh	Mesh để nhìn có thể khác mesh để va chạm.	Bật hiển thị colliders để xem collision shape.
Mass/Inertia	Ảnh hưởng đến ổn định vật lý.	Vật quá nhẹ/nặng gây rung hoặc bay.
Joint/Drive	Khớp và bộ điều khiển joint trong articulation.	Joint drive type/limit/stiffness/damping hợp lý không?
Simulation timestep	Bước thời gian physics.	Timestep quá lớn có thể làm va chạm kém ổn định.
Contact	Tương tác tiếp xúc giữa gripper, object, table, foam/carton.	Có collision pair hợp lý không? Có xuyên vật không?

Debug physics: Khi vật thể cứ xử lạ, kiểm tra collider trước khi sửa AI. Nên bật hiển thị physics/collider

trong viewport, xem log PhysX và chạy một scene nhỏ để tái hiện lỗi.

3.8. Camera và sensor: đầu vào của perception

Perception trong các task HRC2026 phụ thuộc vào camera/sensor của scene. Ở Module 2, sinh viên chưa cần viết thuật toán pose estimation, nhưng phải biết camera ở đâu, nhìn cái gì, output được lấy theo cơ chế nào và có những modality nào có thể dùng.

Dữ liệu sensor	Dùng để làm gì trong HRC2026	Câu hỏi cần trả lời
RGB image	Nhận dạng loại workpiece, nhìn carton/foam, debug scene.	Camera nào? Resolution? Có object target trong ảnh không?
Depth	Tính khoảng cách/3D point, hỗ trợ pose/grasp.	Depth unit là gì? Có bị clipping không?
Segmentation/labels	Tạo synthetic label hoặc debug perception.	Object đã có semantic label chưa? Annotator nào đang bật?
Bounding boxes 2D/3D	Tạo nhãn hoặc kiểm tra detector.	Có đồng bộ với frame RGB không?
Camera pose/extrinsic	Chuyển từ camera frame sang world/robot frame ở Module 3-4.	Camera prim path và transform là gì?

Mẫu camera checklist

```
# Checklist camera cho Module 2
Camera prim path:
Resolution:
RGB output path/topic/buffer:
Depth output path/topic/buffer:
Camera nhìn thấy: table / workpieces / bins / conveyor / foam / carton
Known issue: black image / clipping / wrong angle / low FPS / missing labels
```

3.9. Robot articulation và điều khiển ở mức khái niệm

Trong Isaac Sim, robot nhiều khớp thường được mô phỏng như một articulation. Ở Module 2, chưa cần giải IK, nhưng sinh viên cần hiểu rằng mọi command xuống robot cuối cùng phải khớp với joint names/indices, action dimension, controller interface và robot root/articulation path.

Khái niệm	Ý nghĩa thực dụng	Lỗi nếu hiểu sai
Articulation root	Prim gốc của cơ cấu robot nhiều khớp.	Controller không tìm thấy robot hoặc không initialize được.
Joint names/indices	Tên/thứ tự khớp được controller dùng.	Action gửi nhầm khớp, tay trái/phải đảo nhau.
Position/velocity/effort command	Các kiểu điều khiển joint.	Dùng đồng thời nhiều control mode gây xung đột.

Initialize before command	Robot/controller phải được initialize sau khi simulation/world sẵn sàng.	apply_action không có tác dụng hoặc báo lỗi.
Gripper interface	Open/close hoặc joint command riêng cho gripper.	Gắp không đóng/mở đúng hoặc giữ vật không ổn định.

3.10. Performance, cache và headless evaluation

Vấn đề	Dấu hiệu	Hướng xử lý ban đầu
GPU memory cao	Isaac Sim crash hoặc FPS thấp khi render.	Giảm resolution, đóng ứng dụng khác, dùng headless khi batch eval.
Shader/asset cache lâu	Lần đầu mở scene rất chậm.	Không xóa cache nếu không cần; chuẩn hóa cache chung trong Module 1.
Headless khác GUI	GUI chạy được nhưng batch/headless lỗi.	Kiểm tra extension load, camera/render product và path khi không có viewport.
FPS thấp	Control loop không real-time, policy chậm.	Log thời gian từng bước: perception, policy, action, world.step.
Không tái lập	Mỗi lần chạy kết quả khác quá nhiều.	Ghi seed, config, commit, checkpoint và version assets.

3.11. Quy trình debug Isaac Sim theo 7 câu hỏi

15. Lỗi xảy ra ở GUI, standalone/headless hay cả hai?
16. Scene/asset đã load đủ chưa? Có missing reference, missing mesh, Git LFS issue không?
17. Prim path robot/object/camera có đúng theo Stage hiện tại không?
18. Physics đã Play/initialize chưa? Collider/rigid body có đúng không?
19. Robot articulation/controller đã initialize chưa? Joint names/action dimension đúng không?
20. Camera/sensor có output hợp lệ không? RGB/depth có đồng bộ và nhìn đúng vùng không?
21. Log lỗi chỉ ra file/config/function nào? Có thể tái hiện bằng script tối giản không?

3.12. Lộ trình tự học Isaac Sim trong 1 ngày cho sinh viên mới

Thời lượng	Nội dung	Deliverable
30 phút	Xem overview Isaac Sim và đọc UI reference: Menu, Viewport, Stage, Toolbar.	Ghi 10 thuật ngữ UI.
60 phút	Làm quickstart hoặc course "Building Your First Robot".	Ảnh scene có object/robot đơn giản.

60 phút	Mở HRC2026 scene/assets, lập Stage/prim map.	scene_map_task1.md.
60 phút	Đọc workflow GUI vs Extension vs Standalone Python.	Bảng “khi nào dùng workflow nào”.
60 phút	Đọc URDF import và robot simulation/articulation ở mức khái niệm.	robot_structure_note.md bản nháp.
60 phút	Đọc camera/synthetic data docs ở mức biết RGB/depth/segmentation.	camera_checklist.md.
60 phút	Chạy/đọc baseline command và log lỗi nếu có.	code_reading_note.md + questions.md.

4. OpenUSD: stage, prim, path, layer, reference, Xform

OpenUSD là nền tảng mô tả scene 3D mà Isaac Sim sử dụng. Khi sinh viên nghe các từ như stage, prim, Xform, Mesh, reference hoặc layer, hãy hiểu đây là ngôn ngữ dùng để mô tả scene, không phải thuật toán AI. Nắm được USD giúp nhóm giao tiếp chính xác: “camera prim path sai”, “object được reference từ asset”, “robot root Xform ở path này”, thay vì nói chung chung “scene lỗi”.

Thuật ngữ	Giải thích ngắn	Ví dụ trong HRC2026
Stage	Toàn bộ scene sau khi USD compose các layer/reference.	Một task scene chứa robot, bàn, vật, camera, ánh sáng.
Prim	Đơn vị cơ bản trong scene graph. Prim có path, type và properties.	Camera, mesh vật thể, table, robot root.
Prim path	Đường dẫn duy nhất đến prim trong stage.	/World/Robot/right_arm/... hoặc path thực tế.
Xform	Prim chứa transform, dùng để nhóm/di chuyển các prim con.	Root của một object hoặc cụm robot.
Mesh/Gprim	Hình học render được.	STL/mesh của workpiece, table, box.
Camera	Prim sensor camera trong USD/Isaac Sim.	Camera RGB-D dùng cho perception.
Layer	Một file USD hoặc layer trong quá trình compose scene.	Scene chính có thể tham chiếu asset robot và asset bàn.
Reference	Cách một USD file tham chiếu	Task scene reference WalkerS2

	asset từ file khác.	USD hoặc box/conveyor assets.
Articulation	Robot nhiều khớp được mô phỏng như một cơ cấu khớp nối.	Walker S2 khi được nạp vào Isaac Sim.

4.1. Cách đọc một USD stage

Khi mở một scene, sinh viên nên đọc theo thứ tự từ tổng quát đến chi tiết:

22. Xác định root prim chính: thường là /World hoặc tương đương.
23. Tìm robot root và kiểm tra nó có phải articulation/robot model hay không.
24. Tìm các nhóm scene element: table, box, conveyor, foam, carton, bins, workpieces.
25. Tìm camera/sensor prim đang được code đọc.
26. Kiểm tra path nào là asset reference, path nào là object spawn trong task.
27. Ghi lại các path quan trọng vào scene map và config checklist.

Script mẫu: list_usd_prims.py

```
# Script mẫu liệt kê prim trong một file USD
# Chạy trong môi trường có pxx/Usd, thường là Python environment của Isaac Sim.
from pxx import Usd
import sys

usd_file = sys.argv[1]
stage = Usd.Stage.Open(usd_file)
for prim in stage.Traverse():
    print(f"{prim.GetPath()} {prim.GetTypeName()}")

# Ví dụ chạy:
# python list_usd_prims.py /path/to/scene.usd > scene_prims.txt
```

4.2. Quy ước đặt tên và lỗi hay gặp với prim path

Lỗi	Triệu chứng	Cách kiểm tra
Sai path	Code báo không tìm thấy prim, camera hoặc object None.	Copy path trực tiếp từ Stage hoặc dùng script Traverse.
Tên có ký tự đặc biệt bị thay đổi	URDF import sang USD đổi dấu gạch/ký tự lạ thành underscore.	So sánh URDF link name và USD prim name sau import.
Object nằm trong reference/payload	Không thấy mesh con nếu chưa load đúng layer/reference.	Kiểm tra asset path và layer/reference trong USD.
Scale/up-axis sai	Object quá to/nhỏ hoặc xoay lệch.	Kiểm tra units, scale, transform và up axis.

5. USD, URDF và STL: khác nhau ở đâu và dùng khi nào

Trong tài nguyên HRC2026, sinh viên sẽ gặp nhiều loại file. Ba loại thường gây nhầm lẫn nhất là USD, URDF và STL. Hãy xem chúng như ba lớp mô tả khác nhau: USD mô tả scene và asset trong Isaac Sim; URDF mô tả cấu trúc robot; STL mô tả hình học bề mặt của một phần tử.

Định dạng	Mục đích chính	Chứa gì	Không nên kỳ vọng gì
USD / USDA / USDC / USDZ	Mô tả scene/asset trong OpenUSD/Isaac Sim.	Stage, prims, transforms, materials, cameras, lights, references, physics schemas, robot asset sau import.	Không phải lúc nào cũng là file duy nhất; có thể reference nhiều asset khác.
URDF	Mô tả robot theo XML, thường dùng trong ROS/robotics.	Links, joints, parent-child tree, visual/collision/inertial, joint limits, mesh references.	Không mô tả đầy đủ scene phức tạp như USD; không tự chứa mọi mesh nếu dùng file tham chiếu.
STL	Mô tả mesh hình học 3D.	Bề mặt tam giác của một bộ phận.	Không có joint, material nâng cao, semantics robot, controller hoặc task logic.

5.1. URDF ở mức cần đọc

URDF là XML. Sinh viên chưa cần viết URDF mới, nhưng cần đọc được các thành phần cơ bản sau để hiểu model Walker S2:

Thẻ URDF	Ý nghĩa	Câu hỏi cần trả lời
<robot>	Root của mô tả robot.	Tên robot là gì? File có include/xacro không?
<link>	Một phần cứng/rigid body của robot.	Link nào là base, torso, cánh tay, gripper, camera mount?
<joint>	Khớp nối giữa parent link và child link.	Joint type là revolute/prismatic/fixe? Parent-child là gì?
<visual>	Mesh/hình ảnh để render.	Mesh file nằm ở đâu? Scale có hợp lý không?
<collision>	Mesh/hình collision.	Collision có đơn giản hơn visual không?
<inertial>	Khối lượng, inertia.	Có thiếu inertial khiến physics không ổn định không?
<limit>	Giới hạn joint.	Joint có lower/upper/velocity/effort không?

Script mẫu: `list_urdf_structure.py`

```
# Script mẫu đọc link và joint từ URDF
import xml.etree.ElementTree as ET
import sys

urdf_file = sys.argv[1]
root = ET.parse(urdf_file).getroot()

print('LINKS')
for link in root.findall('link'):
    print('-', link.attrib.get('name'))

print('JOINTS')
for joint in root.findall('joint'):
    name = joint.attrib.get('name')
    jtype = joint.attrib.get('type')
    parent = joint.find('parent').attrib.get('link') if joint.find('parent') is not None else '?'
    child = joint.find('child').attrib.get('link') if joint.find('child') is not None else '?'
    print(f"- {name}: {jtype}, {parent} -> {child}")

# Ví dụ chạy:
# python list_urdf_structure.py WalkerS2.urdf > walker_s2_urdf_note.txt
```

5.2. Import URDF sang USD trong Isaac Sim

Khi một robot URDF được đưa vào Isaac Sim, Isaac Sim sẽ chuyển robot thành asset USD để mô phỏng. Quá trình này không chỉ là “đổi đuôi file”; nó liên quan đến mesh, scale, collision, joint drive, articulation root, naming convention và asset reference. Vì vậy, khi gặp lỗi robot không hoạt động, phải kiểm tra cả URDF gốc, USD đã import và config/controller đang dùng.

- [] URDF có đầy đủ mesh path không?
- [] Mesh có scale đúng đơn vị mét không?
- [] Joint limits có hợp lý không?
- [] Collision meshes có tồn tại và không quá phức tạp không?
- [] Sau import, joint/link names có bị đổi ký tự không?
- [] Robot root/articulation root có đúng path không?
- [] Camera/gripper/end-effector có nằm dưới robot tree đúng không?

6. Đọc model Walker S2: link, joint, camera, end-effector

Walker S2 là robot dùng trong vòng mô phỏng. Ở Module 2, sinh viên chưa cần điều khiển robot tinh vi; nhiệm vụ là hiểu robot được tổ chức trong file model và scene. Tất cả tên path/joint/link phải được kiểm chứng từ asset hiện tại, không đoán theo hình ảnh.

Nhóm thông tin	Cần ghi lại trong robot structure note	Vì sao quan trọng
Robot root/articulation root	Path của robot root trong USD stage; path root trong URDF nếu có.	Controller và physics cần biết root để điều khiển robot.
Base/torso/waist	Link gốc, link thân, joint eo nếu có.	Cuộc thi khóa thân, chỉ dùng eo và hai tay; cần biết joint nào

		được phép.
Right arm/left arm	Danh sách joint/link theo từng tay, thứ tự từ vai đến cổ tay.	IK và action vector thường phụ thuộc đúng thứ tự joint.
Grippers	Link/joint gripper, control interface, open/close convention nếu có.	Gắp/nhả vật phụ thuộc gripper.
End-effector candidates	Link cuối cổ tay/gripper hoặc frame dùng làm target pose.	Motion primitive cần target end-effector.
Camera/sensors	Camera prim path, mounting link, RGB/depth output nếu có.	Perception cần biết camera nào cung cấp ảnh.
Joint limits	Lower/upper/velocity/effort nếu đọc được.	Tránh gửi action vượt giới hạn.

Không được nhảm hình học với điều khiển: Một mesh bàn tay trong viewport không đảm bảo đó là frame end-effector dùng trong code. End-effector phải được xác định từ robot model, controller hoặc baseline wrapper.

6.1. Template Robot Structure Note

Mẫu file: `robot_structure_note.md`

```
# Robot Structure Note - Walker S2
Người thực hiện:
Ngày:
Repo/assets version hoặc commit:

1. File/path đã đọc
- URDF:
- USD:
- Scene/task:

2. Robot root
- USD prim path:
- Articulation root path:
- Base/root link:

3. Waist/torso
- Waist joint(s):
- Locked/fixed joints cần lưu ý:

4. Right arm
- Joint order:
- Link order:
- End-effector candidate:
- Gripper joint/control:

5. Left arm
- Joint order:
- Link order:
```

- End-effector candidate:
- Gripper joint/control:

6. Camera/sensors

- Camera prim path(s):
- Mounting link nếu biết:
- Ghi chú RGB/depth:

7. Câu hỏi chưa rõ

- ...

6.2. Cách kiểm tra thông tin

Cách kiểm tra	Dùng khi nào	Kết quả cần lưu
Isaac Sim Stage panel	Khi cần copy path của robot/camera/object trong scene.	Danh sách prim path có ảnh minh họa nếu cần.
Properties panel	Khi cần xem transform, physics, articulation/joint properties.	Thông số transform/physics quan trọng.
Đọc URDF bằng text/Python	Khi cần link/joint tree và joint limits.	Danh sách link/joint theo nhóm.
Đọc baseline code	Khi cần biết code dùng link/joint/camera nào thật sự.	Dòng code hoặc config key liên quan.
Log khi chạy scene	Khi có lỗi missing asset, invalid path, extension, physics warning.	Lỗi gốc và cách xử lý.

7. Baseline HRC2026: kiến trúc thư mục và luồng chạy

Baseline không chỉ là “mã mẫu”. Nó là khung kỹ thuật kết nối Isaac Sim, Walker S2, task config, data collection, policy training và evaluation. Trong Module 2, sinh viên cần đọc baseline ở mức kiến trúc trước khi sửa thuật toán. Mục tiêu là biết entrypoint nào chạy gì, config nào quyết định scene/task, dữ liệu đi từ đâu đến đâu.

7.1. Bản đồ logic của baseline

HRC2026 baseline - bản đồ logic nên hiểu

```
assets/models/configs
-> Isaac Sim scene/env
-> robot wrapper / articulation / controller interface
-> observation builder: camera + robot state + task state
-> human teleoperation hoặc policy/planner
-> action command: joint/EE/gripper/waist
-> physics step trong Isaac Sim
-> logger: video, state, action, metadata
-> training/evaluation scripts
```


Lớp kiến trúc	Cần tìm trong repo	Câu hỏi đọc code
Environment/scene	Folder asset, scene, task config, Isaac Sim launcher.	Scene nào được load? Robot và objects xuất hiện từ đâu?
Robot interface	Wrapper cho Walker S2, articulation, controller, IK/gripper nếu có.	Action được gửi vào robot theo format nào? Joint nào được điều khiển?
Task logic	Task config, reset/spawn, success/failure, bin/foam/carton positions.	Task tạo vật như thế nào? Goal nằm ở đâu?
Data collection	Teleoperation/record/replay scripts.	Observation/action/metadata được ghi ra đâu? Tần số bao nhiêu?
Training/policy	LeRobot integration, train script, policy configs.	Dataset nào được dùng? Checkpoint lưu đâu?
Evaluation	Eval script, metric/logging.	Success/time/accuracy được đo bằng cách nào?

7.2. Lệnh đọc repo hiệu quả

Lệnh gợi ý khi đọc baseline

```
# 1. Xem cấu trúc repo ở mức vừa đủ
pwd
tree -L 2 . 2>/dev/null || find . -maxdepth 2 -type d | sort

# 2. Tìm file config theo tên task hoặc keyword
find . -iname "*sorting*" -o -iname "*conveyor*" -o -iname "*packing*" -o -iname "*foam*"

# 3. Tìm entrypoint Python thường gặp
find . -maxdepth 4 -type f -name "*.py" | sort | sed -n '1,200p'

# 4. Dùng grep nếu chưa có ripgrep
# grep -R "control_robot\|teleop\|record\|replay\|train\|eval" -n . | head -100

# 5. Nếu có ripgrep
rg "control_robot|teleop|record|replay|train|eval|Part_Sorting|Conveyor|Packing|Foam" .
```

7.3. Code reading note

Mẫu file: code_reading_note.md

```
# Code Reading Note - HRC2026 Baseline
Người thực hiện:
Ngày:
Repo commit/hash nếu có:

1. Entry points
- Script chạy simulation:
```

- Script teleoperation/record:
 - Script replay:
 - Script train:
 - Script inference/eval:
2. Folder quan trọng
- assets/models:
 - configs:
 - Ubtech_sim hoặc module simulation:
 - lerobot hoặc integration:
 - datasets:
 - scripts:
3. Data flow
- Observation gồm:
 - Action format gồm:
 - Logger ghi:
 - Checkpoint/model nằm ở:
4. Task config
- Task 1 config:
 - Task 2 config:
 - Task 3 config:
 - Task 4 config:
5. Những dòng code cần đọc kỹ
- File/path:
 - Hàm/class:
 - Ghi chú:
6. Rủi ro hoặc câu hỏi
- ...

Thói quen bắt buộc: Mọi kết luận về baseline phải kèm đường dẫn file hoặc dòng code. Không ghi “em nghĩ script A dùng camera B”; hãy ghi “file X, function Y, config key Z cho thấy camera path là ...”.

8. Config theo task: đọc đúng trước khi sửa

Task config là nơi liên kết thuật toán với scene. Sinh viên thường muốn sửa model hoặc code ngay, nhưng nhiều lỗi đến từ config: scene sai, asset path sai, camera path sai, object spawn sai, bin/foam/carton target sai, hoặc policy checkpoint trở nhầm. Trước khi viết code task, mỗi nhóm phải có config checklist.

Task	Scene cần hiểu	Config/key cần tìm	Output Module 2
Task 1 - Desktop Sorting	Bàn, 4 workpieces, 2 loại, bins phân vùng.	Object spawn, class labels, bin positions, camera path, robot path.	Task1 config checklist.
Task 2 - Conveyor Sorting	Băng chuyền, vật chuyển động tốc độ cố định, 2 bins.	Conveyor path, velocity, spawn schedule, tracking camera, bins, allowed workspace.	Task2 config checklist.

Task 3 - Workpiece Insertion	Material box, 6 workpieces, foam ở giữa bàn.	Foam path, slot positions, workpiece types/counts, material box pose, camera path.	Task3 config checklist.
Task 4 - Carton Sealing	Carton có foam/workpieces bên trong, nắp ngắn và nắp dài.	Carton/lid path, lid ordering, dual-arm sequence, camera path, success check nếu có.	Task4 config checklist.

8.1. Template Config Checklist

Mục	Câu hỏi cần trả lời	Giá trị thực tế / file / path
Task name	Tên task và config file là gì?	
Scene file	Scene/USD nào được load?	
Robot model	Robot USD/URDF path nào được dùng?	
Robot root	Robot root/articulation path là gì?	
Camera	Camera RGB/depth path nào được dùng?	
Target objects	Object types, số lượng, spawn rules?	
Goal area	Bins/foam/carton target path hoặc pose?	
Controller/policy	Scripted/teleop/policy nào đang được dùng?	
Logging	Log/dataset/checkpoint lưu ở đâu?	
Known issues	Lỗi hoặc điểm chưa rõ?	

8.2. Quy trình đọc một config

28. Tìm config file từ README hoặc command line chạy task.
29. Đọc các key về scene/asset trước, chưa sửa tham số model.
30. Mở scene tương ứng trong Isaac Sim và xác minh path.
31. So sánh config với code: key nào được đọc ở hàm/class nào.
32. Chạy một smoke test và đối chiếu log với config.
33. Ghi checklist, chỉ sửa khi biết rõ key ảnh hưởng đến đâu.

9. Quy trình code reading và reverse engineering baseline

Reverse engineering ở đây không có nghĩa là phá mã nguồn. Nó là kỹ năng đọc một hệ thống có sẵn để hiểu luồng dữ liệu và điểm cần sửa. Với sinh viên mới, quy trình đọc baseline cần có kỷ luật để tránh lạc vào chi tiết.

Bước	Việc làm	Output
1. Entry command	Từ README hoặc script, xác định lệnh chạy task/simulation/teleop/train/eval.	Danh sách command và tham số.
2. Entrypoint file	Mở file Python/bash được command gọi đầu tiên.	Entry file và main function.
3. Config loading	Tìm đoạn load YAML/JSON/config args.	Config keys quan trọng.
4. Environment creation	Tìm đoạn tạo Isaac Sim app/world/env/task.	Scene/task/env class.
5. Robot creation	Tìm đoạn nạp robot, articulation, controller.	Robot path, controller class.
6. Observation/action loop	Tìm vòng lặp step: observe -> action -> step -> log.	Data flow diagram.
7. Logging/evaluation	Tìm đoạn ghi dataset/log hoặc tính success.	Log schema và metric.

Luồng logic cần nhận diện

```
# Pseudocode cần tìm trong baseline
load_config(args)
start_isaac_sim(config)
world = create_world(config)
robot = load_robot(config.robot)
task = load_task(config.task)
policy_or_teleop = create_controller(config)

while not task.done():
    obs = world.get_observation()
    action = policy_or_teleop(obs)
    robot.apply_action(action)
    world.step()
    logger.write(obs, action, task_state)

metrics = evaluator.compute(logs)
```

10. Các lỗi thường gặp khi đọc scene/model/baseline

Nhóm lỗi	Biểu hiện	Cách xử lý ban đầu
----------	-----------	--------------------

Missing asset/reference	Scene mở nhưng vật thể mất, màu hồng/đen, console báo file not found.	Kiểm tra asset path, Git LFS, symlink, cache, relative path.
Wrong prim path	Code không lấy được camera/object/robot, NoneType error.	Copy path từ Stage hoặc list prim bằng script.
URDF import mismatch	Joint/link name sau import khác URDF gốc.	Kiểm tra quy ước đổi ký tự, so sánh USD prim tree.
Scale/unit mismatch	Robot/vật quá to hoặc quá nhỏ.	Kiểm tra meters, scale, import options, asset transform.
Physics/collision issue	Vật rơi xuyên bàn, robot rung, object bay khỏi scene.	Kiểm tra collider, mass/inertia, simulation step, contact settings.
Camera black/empty	Ảnh camera đen hoặc không thấy object.	Kiểm tra camera pose, clipping, resolution, render mode, camera path.
Config trở nhầm task	Chạy Task 2 nhưng scene hoặc asset Task 1 xuất hiện.	Kiểm tra command, args, config include, default config.
Repo branch lệch	Lệnh chạy theo tài liệu nhưng file không tồn tại.	Ghi commit/branch, đọc README hiện tại, hỏi Platform guild.

11. Lab thực hành từng bước

Lab 1 - Mở scene và lập scene map

Mục tiêu	Sinh viên biết mở scene/task và ghi lại các prim path chính.
Thời lượng	60-90 phút
Input	Baseline đã setup từ Module 1, asset folder hoặc scene task hiện có.
Output	scene_map_task1.md hoặc tương đương.

- ☐ Mở Isaac Sim hoặc chạy scene theo hướng dẫn baseline.
- ☐ Xác định robot root, table, target objects, bins/foam/carton/conveyor nếu có.
- ☐ Copy ít nhất 10 prim path quan trọng.
- ☐ Chụp 1 ảnh viewport hoặc mô tả layout bằng text.
- ☐ Ghi các path vào scene map và commit lên repo/lab folder.

Lab 2 - Liệt kê prim bằng script

Mục tiêu	Sinh viên biết kiểm tra scene không chỉ bằng mắt mà bằng script.
Thời lượng	45-60 phút
Output	scene_prims.txt và ghi chú 10 prim quan trọng.

- ☐ Tạo script list_usd_prims.py từ mẫu trong tài liệu.
- ☐ Chạy script trong Python environment có pxr/Usd.
- ☐ Lọc các prim có type Camera, Xform, Mesh hoặc tên liên quan robot/object.
- ☐ So sánh kết quả script với Stage panel trong Isaac Sim.

Lab 3 - Đọc URDF/robot model

Mục tiêu	Sinh viên đọc được link/joint tree ở mức cơ bản.
Thời lượng	60-90 phút
Output	walker_s2_urdf_note.txt và robot_structure_note.md.

- ☐ Tìm file URDF hoặc robot model của Walker S2 trong assets/repo.
- ☐ Chạy script list_urdf_structure.py hoặc đọc XML bằng text editor.
- ☐ Ghi lại các link/joint liên quan đến torso/waist/right arm/left arm/gripper.
- ☐ Đánh dấu các câu hỏi chưa rõ thay vì tự đoán.

Lab 4 - Đọc baseline architecture

Mục tiêu	Sinh viên đọc được entrypoint và data flow của baseline.
Thời lượng	90-120 phút
Output	code_reading_note.md.

- ☐ Chạy tree/find/rg để xem cấu trúc repo.
- ☐ Xác định entrypoint chạy simulation/teleop/record/replay/train/eval.
- ☐ Tìm các file config task.
- ☐ Vẽ sơ đồ data flow: config -> scene -> robot -> observation -> action -> log.
- ☐ Ghi ít nhất 5 file quan trọng và mục đích của từng file.

Lab 5 - Config checklist cho 4 task

Mục tiêu	Sinh viên có checklist dùng lại trước khi triển khai task.
Thời lượng	90 phút
Output	task_config_checklist.xlsx hoặc

	task_config_checklist.md.
--	---------------------------

- ☐ Tạo bảng checklist theo template ở mục 8.1.
- ☐ Điền tối thiểu cho Task 1 và Task 2; nếu đủ thời gian điền cả Task 3, Task 4.
- ☐ Với mỗi giá trị quan trọng, ghi nguồn: file config, dòng code, hoặc prim path từ scene.
- ☐ Đánh dấu các mục chưa rõ để hỏi mentor hoặc Platform guild.

12. Mini-project cuối module: HRC2026 Scene & Baseline Explorer

Mini-project này kiểm tra năng lực tự học Module 2. Mỗi sinh viên làm cá nhân hoặc theo cặp. Platform và Manipulation guild nên làm sâu hơn; các guild khác vẫn phải hoàn thành bản tối thiểu để hiểu hệ thống chung.

Tên mini-project	HRC2026 Scene & Baseline Explorer
Thời lượng	4-6 giờ làm cá nhân hoặc 6-8 giờ theo cặp.
Mục tiêu	Tạo tài liệu kỹ thuật giúp một thành viên mới hiểu scene, robot model, baseline architecture và task configs.
Deliverable chính	Một thư mục nộp gồm README, scene map, robot structure note, code reading note, config checklist, script inspect nếu có.

Cấu trúc thư mục nộp mini-project

```
module2_submission_<student_id>/
  README.md
  scene_map_task1.md
  scene_map_task2_optional.md
  robot_structure_note.md
  code_reading_note.md
  task_config_checklist.md
  scripts/
    list_usd_prims.py
    list_urdf_structure.py
  screenshots/
    scene_overview.png
    robot_tree_or_stage.png
  questions.md
```

12.1. Yêu cầu chi tiết

- ☐ README.md mô tả cách bạn chạy/mở scene và commit/branch đang dùng.
- ☐ Scene map có ít nhất 10 prim path quan trọng và mô tả vai trò của từng path.
- ☐ Robot structure note có robot root, arm/gripper/end-effector candidates, camera/sensor nếu tìm được.
- ☐ Code reading note nêu entrypoint, folder quan trọng, data flow và 5 file quan trọng.
- ☐ Config checklist có ít nhất Task 1 và Task 2; khuyến khích thêm Task 3 và Task 4.
- ☐ questions.md ghi những điểm chưa chắc chắn, mỗi câu hỏi phải có bối cảnh rõ.

12.2. Rubric chấm mini-project

Tiêu chí	Điểm	Mô tả mức đạt tốt
Scene map	20	Path chính xác, có phân nhóm robot/objects/camera/task elements, có ảnh hoặc mô tả layout rõ.
Robot structure note	20	Ghi được root, arm/gripper/camera/end-effector candidates; phân biệt điều đã xác minh và điều chưa chắc.
Baseline architecture note	20	Xác định entrypoints, folders, data flow, config loading, observation-action-log loop.
Task config checklist	15	Checklist có nguồn rõ: config/file/path; không điền suy đoán.
Inspect scripts/logs	10	Có script hoặc lệnh tái lập để liệt kê prim/link/joint hoặc đọc repo; có log/output mẫu.
Clarity & reproducibility	10	Người khác đọc được, chạy lại được, ghi commit/branch/version.
Teamwork/issue quality	5	Câu hỏi rõ ràng, biết báo lỗi bằng evidence, không ghi chung chung.
Tổng	100	Dùng để chọn guild và đánh giá readiness cho Module 3-4.

12.3. Mức đánh giá

Mức	Điểm	Diễn giải
Excellent	85-100	Có thể trở thành tài liệu dùng chung cho guild; path và code notes có evidence.
Good	70-84	Hoàn thành đủ, còn thiếu một số chi tiết nhưng dùng được để học tiếp.
Pass	60-69	Hiểu cơ bản nhưng cần mentor/guild hỗ trợ trước khi làm

		task thật.
Needs work	<60	Chưa đủ để tự đọc scene/baseline; cần làm lại lab trước Module 3.

13. Checklist hoàn thành Module 2

- ☐ Tôi giải thích được stage, prim, prim path, Xform, Mesh, Camera, Layer, Reference bằng lời của mình.
- ☐ Tôi phân biệt được USD, URDF và STL, biết file nào nên đọc khi gặp lỗi scene/robot/mesh.
- ☐ Tôi mở được một scene hoặc ít nhất đọc được file scene/asset và liệt kê prim path quan trọng.
- ☐ Tôi tạo được scene map cho ít nhất một task.
- ☐ Tôi tạo được robot structure note cho Walker S2 ở mức cơ bản.
- ☐ Tôi biết tìm entripoint chạy simulation/teleop/record/train/eval trong baseline.
- ☐ Tôi có code reading note với data flow observe -> action -> step -> log.
- ☐ Tôi điền được config checklist cho ít nhất Task 1 và Task 2.
- ☐ Tôi biết cách báo lỗi scene/model/config bằng log, path và file cụ thể.

13.1. Câu hỏi tự kiểm tra

34. Stage khác gì với prim? Cho ví dụ một prim trong HRC2026.
35. Vì sao cần prim path chính xác khi viết perception hoặc controller?
36. USD khác URDF ở điểm nào? STL khác hai định dạng trên ở điểm nào?
37. Một object rơi xuyên qua bàn trong simulation có thể do những nguyên nhân nào?
38. Nếu code báo không tìm thấy camera, bạn kiểm tra theo thứ tự nào?
39. Robot structure note cần ghi những nhóm thông tin nào?
40. Tại sao end-effector không nên được xác định chỉ bằng mắt trong viewport?
41. Một task config tốt cần ghi những gì?
42. Khi đọc baseline, vì sao cần tìm entripoint trước khi đọc từng file nhỏ?
43. Hãy vẽ luồng logic từ config đến scene, observation, action và log.

14. Tài liệu tham khảo mở/chính thức

Ưu tiên đọc tài liệu chính thức và repository gốc. Khi có khác biệt giữa tài liệu này và README hiện tại của repository, luôn ưu tiên README/commit hiện tại của repository.

14.1. HRC2026/UBTECH resources

- **UBTECH HRC2026 Baseline GitHub:** https://github.com/UBTECH-Robot/GlobalHumanoidRobotChallenge_2026_Baseline - Repo baseline chính thức: simulation, teleoperation/data recording, training, deployment/evaluation.
- **UBTECH challenge2026_assets Hugging Face:** https://huggingface.co/UBTECH-Robotics/challenge2026_assets - Assets mô phỏng: USD/URDF, Walker S2, table, box, conveyor, foam, task components.
- **UBTECH challenge2026_dataset Hugging Face:** https://huggingface.co/datasets/UBTECH-Robotics/challenge2026_dataset - Dataset chính thức theo format LeRobotDataset dùng để học cấu trúc dữ liệu và baseline training.
- **UBTECH WalkerS2-Model GitHub:** <https://github.com/UBTECH-Robot/WalkerS2-Model> - Model URDF/USD của Walker S2 để học robot description, link/joint và asset structure.

- **UBTECH WalkerS2-Model-Challenge GitHub:** <https://github.com/UBTECH-Robot/WalkerS2-Model-Challenge> - Phiên bản model Walker S2 dành cho môi trường challenge nếu repository được công bố/duy trì.

14.2. NVIDIA Isaac Sim/OpenUSD

- **NVIDIA Isaac Sim Developer Page:** <https://developer.nvidia.com/isaac/sim> - Tổng quan Isaac Sim, simulation, synthetic data, robotics workflows.
- **Isaac Sim Documentation:** <https://docs.isaacsim.omniverse.nvidia.com/5.1.0/index.html> - Tài liệu chính thức cho Isaac Sim 5.1.x.
- **Isaac Sim Robot Simulation:** https://docs.isaacsim.omniverse.nvidia.com/5.0.0/robot_simulation/index.html - Robot simulation, articulation, controllers, sensors, importers/exporters.
- **Isaac Sim URDF Importer Extension:** https://docs.isaacsim.omniverse.nvidia.com/5.1.0/importer_exporter/ext_isaacsim_asset_importer_urdf.html - URDF importer, import options, conventions, log/debug khi import robot.
- **NVIDIA Getting Started with Isaac Sim:** <https://docs.nvidia.com/learning/physical-ai/getting-started-with-isaac-sim/latest/index.html> - Learning modules về robot construction, URDF import, sensors, synthetic data, software-in-the-loop.
- **OpenUSD Glossary:** <https://openusd.org/docs/USD-Glossary.html> - Khái niệm USD: stage, prim, layer, reference, schema, composition.
- **Learn OpenUSD - Prims:** <https://docs.nvidia.com/learn-openusd/latest/stage-setting/prims.html> - Giới thiệu prim là building block của USD scene.
- **Learn OpenUSD - Xform:** <https://docs.nvidia.com/learn-openusd/latest/scene-description-blueprints/xform.html> - Giải thích Xform và transform hierarchy.

14.3. URDF, robot description và LeRobot

- **ROS 2 URDF Tutorial:** <https://docs.ros.org/en/rolling/Tutorials/Intermediate/URDF/URDF-Main.html> - URDF là định dạng XML mô tả geometry và organization của robot trong ROS.
- **ROS urdf API Documentation:** <https://docs.ros.org/en/noetic/api/urdf/html/index.html> - URDF model structure: links, joints, tree structure và parser.
- **Hugging Face LeRobot Documentation:** <https://huggingface.co/docs/lerobot> - Framework robotics/ML dùng cho dataset, policy, training và simulated environments.
- **huggingface/lerobot GitHub:** <https://github.com/huggingface/lerobot> - Code framework, examples và tooling liên quan đến LeRobot.

14.4. Isaac Sim self-paced courses và video tutorial

Các link dưới đây ưu tiên tài liệu/video mở hoặc chính thức. Sinh viên nên học theo thứ tự từ tổng quan, UI, robot asset, sensor, synthetic data, rồi mới đến ROS/OmniGraph nâng cao.

- **NVIDIA Physical AI Learning:** <https://docs.nvidia.com/learning/physical-ai/> - Cổng học self-paced miễn phí về Physical AI, Robotics, Isaac Sim, Isaac Lab và OpenUSD.
- **NVIDIA Robotics Fundamentals Learning Path:** <https://www.nvidia.com/en-us/learn/learning-path/robotics/> - Learning path tổng hợp về robotics foundations, Isaac Sim, Isaac Lab, Isaac ROS và OpenUSD.
- **Getting Started With Isaac Sim:** <https://docs.nvidia.com/learning/physical-ai/getting-started-with-isaac-sim/latest/index.html> - Course/video-based learning path: build robot, import URDF assets, synthetic data, SIL/HIL workflows.
- **Building Your First Robot in Isaac Sim:** <https://docs.nvidia.com/learning/physical-ai/getting-started-with-isaac-sim/latest/building-your-first-robot-in-isaac-sim/index.html> - Module nhập môn có video: giao diện Isaac Sim, tạo robot đơn giản, controller, sensors, accessing data.

- **Ingesting Robot Assets and Simulating Your Robot in Isaac Sim:**
<https://docs.nvidia.com/learning/physical-ai/getting-started-with-isaac-sim/latest/ingesting-robot-assets-and-simulating-your-robot-in-isaac-sim/index.html> - Module học import URDF, chuẩn bị simulation, thêm sensors, truy cập robot đã chuẩn bị.
- **Synthetic Data Generation for Perception Model Training:**
<https://docs.nvidia.com/learning/physical-ai/getting-started-with-isaac-sim/latest/synthetic-data-generation-for-perception-model-training-in-isaac-sim/index.html> - Module học tạo dữ liệu tổng hợp, domain randomization, fine-tuning perception model.
- **Developing Robots With Software-in-the-Loop in Isaac Sim:**
<https://docs.nvidia.com/learning/physical-ai/getting-started-with-isaac-sim/latest/developing-robots-with-sil-in-isaac-sim/index.html> - Module nâng cao hơn về SIL, OmniGraph, scripting Python, ROS 2 và segmentation.
- **NVIDIA YouTube - Isaac Sim: Generating Synthetic Data using Replicator Composer:**
https://www.youtube.com/watch?v=HHzNIh72B_Y - Video NVIDIA Developer về Replicator Composer và synthetic data cho computer vision.
- **NVIDIA YouTube - NVIDIA Isaac Sim: Powering Robotics Development:**
<https://www.youtube.com/watch?v=lcee9ntkOjk> - Video tổng quan ngắn của NVIDIA về Isaac Sim trong phát triển robotics.
- **Third-party beginner video - Day 1 with NVIDIA Isaac Sim:**
<https://www.youtube.com/watch?v=r9CBp5QjtQM> - Video beginner-friendly, không phải tài liệu chính thức; chỉ dùng để làm quen UI/thao tác cơ bản.
- **Third-party ROS 2 video - Using Isaac Sim with ROS2:**
<https://www.youtube.com/watch?v=L1rpxRm0Q1w> - Video không chính thức về ROS 2; chỉ dùng tham khảo sau khi đã nắm Isaac Sim cơ bản.

14.5. Isaac Sim docs theo chủ đề cần đọc trong Module 2

- **Isaac Sim Developer Page:** <https://developer.nvidia.com/isaac-sim> - Trang NVIDIA Developer tổng quan về Isaac Sim: robotics simulation, synthetic data, testing, robot learning.
- **Isaac Sim 5.1 Documentation Index:** <https://docs.isaacsim.omniverse.nvidia.com/5.1.0/index.html> - Trang tài liệu chính thức cho Isaac Sim 5.1.x, phù hợp với baseline khuyến nghị.
- **User Interface Reference:**
https://docs.isaacsim.omniverse.nvidia.com/5.1.0/gui/reference_user_interface.html - Stage, Viewport, Main Toolbar, browsers và các thành phần UI cần biết.
- **Workflows:** <https://docs.isaacsim.omniverse.nvidia.com/5.1.0/introduction/workflows.html> - Ba workflow GUI, Extension và Standalone Python; rất quan trọng khi chuyển từ debug sang headless evaluation.
- **Python Scripting and Tutorials:**
https://docs.isaacsim.omniverse.nvidia.com/5.1.0/python_scripting/index.html - Python scripting, Core API tutorials, scene setup snippets, robot simulation snippets.
- **Python Scripting Concepts:**
https://docs.isaacsim.omniverse.nvidia.com/5.1.0/python_scripting/python_scripting_concepts.html - Standalone vs interactive Python; khi nào dùng script command line và khi nào dùng console/editor.
- **Robot Simulation:** https://docs.isaacsim.omniverse.nvidia.com/5.1.0/robot_simulation/index.html - Joint-level control, controllers cho wheeled robots, manipulators, policy-controlled robots.
- **Articulation Controller:**
https://docs.isaacsim.omniverse.nvidia.com/5.1.0/robot_simulation/articulation_controller.html - Joint position/velocity/effort control, joint names/indices, apply_action; cần đọc trước Module 6.
- **Tutorial: Import URDF:**
https://docs.isaacsim.omniverse.nvidia.com/5.1.0/importer_exporter/import_urdf.html - Tutorial chuyển URDF sang USD, import qua GUI/Python/ROS 2 node.

- **URDF Importer Extension:** https://docs.isaacsim.omniverse.nvidia.com/5.1.0/importer_exporter/ext_isaacsim_asset_importer_urdf.html - Extension importer, import options, Output Log, lỗi/warnings khi import robot.
- **Synthetic Data Recorder:** https://docs.isaacsim.omniverse.nvidia.com/5.1.0/replicator_tutorials/tutorial_replicator_recorder.html - GUI extension ghi synthetic data; hữu ích cho guild Perception/Data.
- **ROS 2 Tutorials:** https://docs.isaacsim.omniverse.nvidia.com/5.1.0/ros2_tutorials/index.html - Không bắt buộc ở vòng đầu, nhưng hữu ích để hiểu bridge, camera, joint control, transform trees.

Phụ lục A. Bảng thuật ngữ nhanh

Thuật ngữ	Nghĩa ngắn
Asset	Tài nguyên dùng trong scene: model robot, mesh, material, texture, object, environment.
Articulation	Cơ cấu nhiều khớp được mô phỏng trong physics engine, ví dụ robot.
Collider	Hình học dùng cho va chạm vật lý, có thể khác mesh hiển thị.
Config	File tham số quyết định scene, robot path, task, camera, policy, logging.
End-effector	Frame/link ở cuối tay robot dùng làm mục tiêu thao tác.
Layer	Lớp USD đóng góp vào scene thông qua composition.
Prim	Đơn vị trong USD scene graph, có path và type.
Reference	Cách USD gọi asset từ file khác vào scene.
Scene map	Tài liệu liệt kê các object/path quan trọng trong scene.
Stage	Scene USD đã được compose và hiển thị/chạy trong Isaac Sim.
URDF	XML mô tả robot bằng links và joints.
USD	Định dạng scene/asset trong OpenUSD, nền tảng của Isaac Sim.
Xform	Prim transform, dùng để nhóm và đặt vị trí/hướng/scale.